

Chương 4: Thiết kế vật lý

Lưu trữ, chỉ mục, tối ưu truy vấn và lựa chọn cấu hình vật lý

Nguyen Van Chien

Trường Đại học Giao thông Vận tải

September 12, 2026

Mục tiêu của Chương 4

- ▶ Giải thích dữ liệu được lưu theo khối, bản ghi và tập tin.
- ▶ Phân biệt tổ chức heap, sorted, hashing và phân vùng.
- ▶ Chọn chỉ mục phù hợp với tải công việc.
- ▶ Ước lượng chi phí truy vấn và so sánh thuật toán kết nối.
- ▶ Nhận diện đánh đổi giữa hiệu năng đọc, ghi và dung lượng.

Tư duy chính

Thiết kế vật lý không đổi ý nghĩa dữ liệu. Nó quyết định cách DBMS đọc, ghi và tìm dữ liệu nhanh hơn.

Đầu vào và đầu ra của thiết kế vật lý

Input

Lược đồ logic đã chuẩn hóa
Khóa và ràng buộc
Đặc tả tải công việc



Xử lý

Chọn tổ chức lưu trữ
Chọn chỉ mục
Ước lượng chi phí



Output

DDL vật lý
Chỉ mục, phân vùng
Quy tắc vận hành

Ví dụ đầu vào workload

Truy vấn	Lần/ngày
Tìm nhân viên theo mã	5000
Liệt kê nhân viên theo phòng	800
Liệt kê dự án theo phòng	200
Cập nhật lương hàng loạt	1

Cùng một lược đồ logic có thể cần cấu hình vật lý khác nhau khi workload khác nhau.

Khối, bản ghi và hệ số chặn

Khối / trang dữ liệu

R1: một bản ghi

R2: một bản ghi

R3: một bản ghi

R4: một bản ghi

$bfr = \lfloor B/R \rfloor$ bản ghi trong một khối

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor, \quad b = \left\lceil \frac{r}{bfr} \right\rceil$$

Ví dụ

$B = 4096$ byte, $R = 100$ byte, $r = 10\,000$.

$bfr = 40$, số khối cần lưu $b = 250$.

Chi phí thường được ước lượng bằng số khối phải đọc, vì mỗi lần I/O đọc nguyên cả khối.

Tổ chức tập tin cơ bản

Tổ chức	Tìm theo khóa	Chèn bản ghi	Khi phù hợp
Heap	$O(b)$, quét tuần tự	$O(1)$, chèn cuối	Bảng nhỏ, ghi nhiều, ít lọc theo khóa
Sorted	$O(\log_2 b)$ nếu tìm nhị phân	Chậm do phải giữ thứ tự	Tìm kiếm theo khoảng, báo cáo theo thứ tự
Hash	$O(1)$ trung bình cho so sánh bằng	$O(1)$ trung bình	Tra cứu chính xác theo khóa, không cần range query

Độc đúng ý nghĩa

Không có tổ chức tập tin tốt cho mọi truy vấn. Lựa chọn phụ thuộc vào cách đọc và ghi dữ liệu hằng ngày.

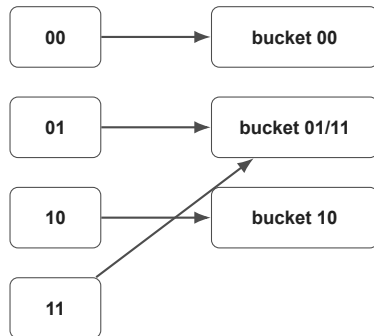
Băm động

Băm mở rộng

Dùng thư mục có độ sâu toàn cục d . Khi ngăn tràn, tách ngăn đó; chỉ nhân đôi thư mục khi độ sâu cục bộ bằng d .

Băm tuyến tính

Tách ngăn theo thứ tự định trước. Không cần thư mục riêng nhưng có thể tồn tại ngăn tràn tạm thời.



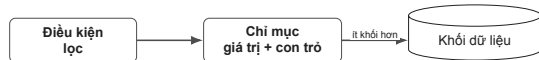
Ví dụ sau khi nhân đôi thư mục: 01 và 11 vẫn có thể trở chung một ngăn.

Chỉ mục và cấu trúc truy xuất

Định nghĩa

Chỉ mục là cấu trúc phụ gồm các mục dạng $\langle gia_tri, con_tro \rangle$, giúp DBMS tìm khối dữ liệu cần đọc mà không quét toàn bộ bảng.

- ▶ Tăng tốc truy vấn đọc.
- ▶ Tốn thêm không gian lưu trữ.
- ▶ Làm chậm ghi dữ liệu vì chỉ mục cũng phải cập nhật.



Ba loại chỉ mục đơn cấp

Loại	Điều kiện	Ví dụ trong CSDL Công ty
Chỉ mục sơ cấp	Tập tin sắp xếp theo khóa chính, chỉ mục thừa	NHANVIEN sắp theo MSNV
Chỉ mục cụm	Tập tin sắp xếp theo cột không khóa, nhiều dòng cùng giá trị	Sắp NHANVIEN theo Ma_phong
Chỉ mục phụ	Không phụ thuộc thứ tự vật lý, thường là chỉ mục dày	Chỉ mục trên Luong, Ho_ten, Ma_duan

Một bảng chỉ có một thứ tự vật lý chính, nên chỉ có tối đa một chỉ mục sơ cấp/cụm theo nghĩa lưu trữ.

Clustered và Nonclustered trong SQL Server

Clustered index

Dữ liệu của bảng được tổ chức theo khóa chỉ mục. Mỗi bảng chỉ có một clustered index.

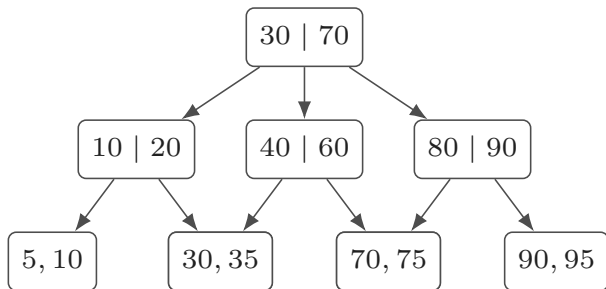
```
CREATE CLUSTERED INDEX IX_NV_MSNV  
ON NHANVIEN(MSNV);
```

Nonclustered index

Chỉ mục riêng trở về dòng dữ liệu. Một bảng có thể có nhiều nonclustered index.

```
CREATE NONCLUSTERED INDEX IX_NV_PHONG  
ON NHANVIEN(Ma_phong);
```

Cây B+



- ▶ Cây cân bằng: mọi đường từ gốc đến lá có cùng độ dài.
- ▶ Nút trong chỉ định hướng tìm kiếm.
- ▶ Nút lá chứa khóa và con trỏ dữ liệu.
- ▶ Các lá nối nhau, hỗ trợ truy vấn theo khoảng.

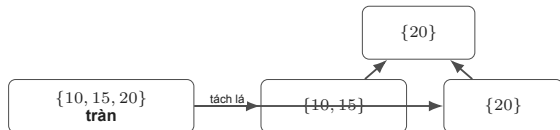
Chi phí tìm kiếm $\approx O(\log_p N)$

Chèn và tách nút trong cây B+

Ví dụ bậc $p = 3$

Lá đang có $\{10, 20\}$. Chèn 15 làm lá tràn vì tối đa 2 khóa. Lá tách thành $\{10, 15\}$ và $\{20\}$.

- ▶ Khóa nhỏ nhất của lá phải được sao chép lên cha.
- ▶ Chiều cao chỉ tăng khi gốc bị tách.



Cây B+ và chỉ mục băm

Tiêu chí	Cây B+	Chỉ mục băm
So sánh bằng	Tốt, $O(\log_p N)$	Rất tốt, $O(1)$ trung bình
Theo khoảng	Tốt nhờ lá nối tiếp	Kém, không giữ thứ tự
Sắp xếp đầu ra	Có thể tận dụng	Không hỗ trợ tự nhiên
Phù hợp	Khóa, khóa ngoại, range query	Tra cứu chính xác theo khóa

Ví dụ

Truy vấn Luong BETWEEN 15000000 AND 25000000 nên dùng B+, không dùng hash.

Algorithm 4.1 Quyết định tạo chỉ mục

- 1: Luôn xem xét khóa chính và khóa ngoại trước.
- 2: Ưu tiên cột xuất hiện thường xuyên trong WHERE, JOIN, ORDER BY, GROUP BY.
- 3: Kiểm tra độ chọn lọc: càng nhiều giá trị phân biệt thì chỉ mục B+ càng hiệu quả.
- 4: Cân nhắc chi phí ghi: nhiều chỉ mục làm chậm INSERT, UPDATE, DELETE.
- 5: Với truy vấn lọc nhiều cột cùng lúc, cân nhắc chỉ mục kết hợp.

Độ chọn lọc của cột

$$selectivity(A) = \frac{NDV(A)}{|R|}$$

Cột	NDV	Dòng	Nhận xét
MSNV	10000	10000	Rất cao
Ma_phong	100	10000	Trung bình
Gioi_tinh	2	10000	Rất thấp

Kết luận

Chỉ mục B+ trên `Gioi_tinh` thường không đáng giá, vì mỗi giá trị trả về quá nhiều dòng.

Chỉ mục kết hợp và thứ tự cột

Chỉ mục

PHANCONG(Ma_duan, MSNV)

- ▶ Hỗ trợ lọc theo Ma_duan.
- ▶ Hỗ trợ lọc đồng thời Ma_duan và MSNV.
- ▶ Không hiệu quả nếu chỉ lọc riêng MSNV.

```
CREATE INDEX IX_PC_DUAN_NV  
ON PHANCONG(Ma_duan, MSNV);
```

```
-- Dung tot  
WHERE Ma_duan = 'DA01'
```

```
-- Khong tan dung tot  
WHERE MSNV = 'NV01'
```

Khai báo chỉ mục cho CSDL Công ty

```
CREATE TABLE PHANCONG (  
  MSNV      varchar(10) NOT NULL,  
  Ma_duan   varchar(10) NOT NULL,  
  So_gio    decimal(5,1) NOT NULL,  
  CONSTRAINT PK_PHANCONG  
    PRIMARY KEY (MSNV, Ma_duan)  
);  
  
CREATE INDEX IX_NV_PHONG  
ON NHANVIEN(Ma_phong);  
  
CREATE INDEX IX_PC_DUAN  
ON PHANCONG(Ma_duan);
```

Chỉ mục	Mục đích
PK_PHANCONG	Tra cứu theo nhân viên và dự án
IX_NV_PHONG	Liệt kê nhân viên theo phòng
IX_PC_DUAN	Liệt kê nhân viên theo dự án

Phân vùng dữ liệu

- ▶ Phân vùng ngang chia các dòng theo khóa phân vùng.
- ▶ Phân vùng dọc chia các cột theo nhóm truy cập.
- ▶ Truy vấn có điều kiện trên khóa phân vùng được cắt tỉa phân vùng.

```
CREATE PARTITION FUNCTION pf_Nam(int)
AS RANGE LEFT FOR VALUES (2023, 2024, 2025);
```

```
CREATE PARTITION SCHEME ps_Nam
AS PARTITION pf_Nam ALL TO ([PRIMARY]);
```

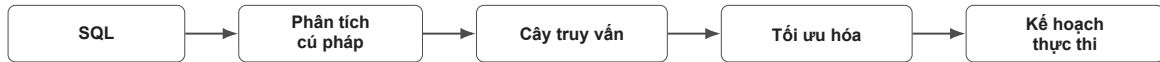
-- Bảng lớn có thể dat trên scheme này

Đánh đổi

Truy vấn trải rộng nhiều phân vùng có thể không nhanh hơn.

Tối ưu hóa truy vấn và ước lượng chi phí

Pipeline xử lý truy vấn



Vai trò của catalog

Bộ tối ưu hóa dựa trên thông kê: số dòng, số khối, độ chọn lọc, chỉ mục hiện có và bộ nhớ khả dụng.

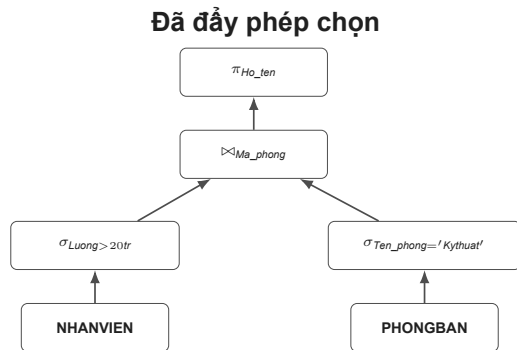
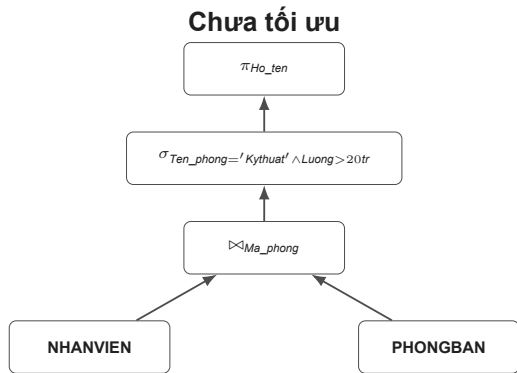
Luật biến đổi tương đương

Luật	Dạng toán
Kết hợp phép chọn	$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_1 \wedge c_2}(R)$
Giao hoán phép chọn	$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$
Giao hoán phép kết	$R \bowtie S \equiv S \bowtie R$
Đẩy chọn qua kết	Nếu c chỉ thuộc R : $\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$

Ý nghĩa

Hai cây truy vấn cho cùng kết quả có thể có chi phí rất khác nhau.

Cây truy vấn trước và sau tối ưu



Lọc **PHONGBAN** và **NHANVIEN** trước khi kết giúp giảm kích thước trung gian.

Chi phí kết nối lồng nhau

$$Cost_{NLJ} \approx b_R + b_R \times b_S$$

Ví dụ chưa lọc

NHANVIEN: $b_R = 250$ khối.

PHONGBAN: $b_S = 5$ khối.

Chi phí: $250 + 250 \times 5 = 1500$.

Sau khi lọc phòng ban

Nếu PHONGBAN còn 1 khối:

Chi phí: $250 + 250 \times 1 = 500$.

Chỉ đổi thứ tự xử lý đã giảm ba lần.

Ba thuật toán kết nối

Thuật toán	Chi phí chính	Điều kiện	Khi dùng
Nested-Loop	$b_R + b_R b_S$	Gần như mọi điều kiện	Một bảng rất nhỏ hoặc có chỉ mục tốt trên bảng trong
Sort-Merge	$O(b_R \log b_R + b_S \log b_S)$, hoặc $O(b_R + b_S)$ nếu đã sắp	Equi-join	Hai bảng đã có thứ tự hoặc cần kết quả có thứ tự
Hash Join	$O(b_R + b_S)$ nếu đủ bộ nhớ	Equi-join	Không có thứ tự sẵn, bảng nhỏ vừa bộ nhớ để build

DBMS thường tự chọn thuật toán dựa trên thống kê và tài nguyên hiện có.

Ước lượng kích thước trung gian

Phép chọn

Với bảng R , cột A có $NDV(A)$ giá trị:

$$|\sigma_{A=c}(R)| \approx \frac{|R|}{NDV(A)}$$

Phép kết

Với phép kết trên A :

$$|R \bowtie S| \approx \frac{|R| \times |S|}{\max(NDV_R(A), NDV_S(A))}$$

Lưu ý

Công thức dựa trên giả định phân bố đều. Dữ liệu lịch cần histogram và thống kê cập nhật.

Case study: chọn chỉ mục theo workload

Truy vấn	Lần/ngày	Quyết định vật lý
Tra cứu nhân viên theo MSNV	5000	Dùng chỉ mục khóa chính
Nhân viên theo Ma_phong	800	Tạo chỉ mục phụ trên NHANVIEN.Ma_phong
Dự án theo Ma_phong	200	Tạo chỉ mục trên DUAN.Ma_phong
Nhân viên tham gia một dự án	300	Tạo chỉ mục trên PHANCONG.Ma_duan
Thống kê theo Gioi_tinh	2	Không tạo B+ thông thường

Bài học

Thiết kế vật lý bắt đầu từ câu hỏi: truy vấn nào lặp lại đủ nhiều để đáng trả chi phí lưu trữ và bảo trì chỉ mục?

Tầng lưu trữ và bài tập vận dụng

RAID và đánh đổi lưu trữ

RAID	Cách tổ chức	Dung lượng với 4 đĩa 2TB	Ghi chú
RAID 0	Phân dải	8TB	Nhanh nhưng hỏng 1 đĩa mất dữ liệu
RAID 1	Nhân bản	4TB	Tin cậy hơn, tốn dung lượng
RAID 5	Phân dải kèm parity	6TB	Chịu hỏng 1 đĩa, có hình phạt ghi

Lưu ý

RAID giúp chịu lỗi phần cứng. Backup vẫn cần thiết để khôi phục khi xóa nhầm, lỗi ứng dụng hoặc mã độc.

Bộ đệm, bitmap và lưu trữ theo cột

Kỹ thuật	Phù hợp	Ghi chú
Buffer pool	Khối được đọc lặp lại	Bảng nhỏ có thể luôn nằm trong RAM
Bitmap index	Cột ít giá trị phân biệt	Tốt cho kho dữ liệu, lọc nhiều điều kiện phân loại
Row-store	Giao dịch đọc/ghi cả dòng	Phù hợp OLTP
Column-store	Truy vấn tổng hợp theo cột	Phù hợp OLAP, nén tốt hơn

```
-- Ví dụ columnstore trong SQL Server
CREATE CLUSTERED COLUMNSTORE INDEX CCI_BAOCao
ON BAOCao_DOANHTHU;
```

Bài tập vận dụng

- 1 Bảng DUAN có 2.000 dòng, bản ghi trung bình 80 byte, khối 4096 byte. Tính *bfr* và số khối cần lưu.
- 2 So sánh chi phí tìm Ma_duan trong heap, sorted file và tập tin có chỉ mục sơ cấp.
- 3 Với cây B+ bậc $p = 3$, lá ban đầu $\{5, 10\}$. Mô tả kết quả sau khi chèn 3 rồi 7.
- 4 Với NHANVIEN 250 khối và THANNHAN 40 khối, so sánh Nested-Loop Join với Hash Join.

Yêu cầu

Mỗi bài cần có công thức, phép tính và kết luận thiết kế vật lý tương ứng.